

Distributed System notes 4

Định danh

Tên, thực thể, điểm truy cập, địa chỉ

- **Tên (Name):** Một chuỗi ký tự hoặc bit dùng để tham chiếu đến thực thể. Mỗi thực thể có thể có nhiều tên.
 - **Thực thể (Entity):** Là bất kỳ đối tượng nào trong hệ thống (ví dụ: máy chủ, máy in, tệp, người dùng, tiến trình).
 - **Địa chỉ (Address):** Là tên của điểm truy cập (access point) — dạng đặc biệt của tên. Một tên có thể ánh xạ đến nhiều địa chỉ; một địa chỉ có thể dẫn đến nhiều thực thể tại các thời điểm khác nhau.
 - **Điểm truy cập (Access point):** Nơi ta truy cập vào thực thể. Một thực thể có thể có nhiều điểm truy cập.
-

Trong máy tính:

- **Name:** Ví dụ như `\\Printer1`, `user123`, hoặc `google.com`, một máy tính
- **Entity:** Có thể là một file (`.txt`), một tiến trình đang chạy, một máy in được chia sẻ, hoặc một user.
- **Address:** Có thể là địa chỉ IP (`192.168.1.10`) hoặc địa chỉ vật lý (MAC).
- **Access Point:** Là nơi tương tác với thực thể, ví dụ: cổng mạng, file descriptor, URL endpoint.

Mục đích trong một hệ phân tán, ta cần tìm được địa chỉ của các máy để kết nối với nhau, xác định lưu dữ liệu vào máy nào. Tuy nhiên, làm thế nào để chọn được một định danh xác định giữa các máy một cách chính xác?

Ví dụ: Một máy tính có thể kết nối với 2 mạng khác nhau nếu nó có 2 interface network

- một Card có thể giao tiếp mạng nội bộ với IP 172.16.x.x
- Một Card giao tiếp với mạng bên ngoài 203.113.2.x

Địa chỉ IP cũng có thể thay đổi theo Wifi, ví dụ, sử dụng wifi trường Phenikaa, địa chỉ IP của máy tính, điện thoại kết nối sẽ thay đổi theo thời gian đăng nhập hoặc timeout

--> Không thể sử dụng IP để làm định danh bởi chúng có thể thay đổi

Định danh

- Identifier(Định danh): một loại tên đặc biệt, phải đáp ứng ba yêu cầu sau:
- 1 định danh chỉ đến nhiều nhất 1 thực thể.
- Mỗi thực thể chỉ được xác định bởi 1 định danh.
- Một định danh mãi mãi chỉ trỏ đến 1 thực thể.

Ví dụ, trong một hệ thống lưu trữ phân tán, định danh giúp xác định **file được lưu trên máy nào**, và sau đó khi cần truy xuất, hệ thống có thể **dựa vào định danh để tìm đúng máy chứa file đó**, thay vì phụ thuộc vào địa chỉ mạng có thể thay đổi theo thời gian.

→ Nói cách khác, **định danh đóng vai trò như một “địa chỉ logic ổn định”**, giúp quản lý và truy cập tài nguyên một cách chính xác và đáng tin cậy trong môi trường mà vị trí vật lý có thể thay đổi linh hoạt.

Vấn đề:

- Làm thế nào để đặt tên định danh không bị trùng
- Cạn kiệt không gian tên (nếu có quá nhiều máy thì đặt như nào nếu bị đầy)

Giải pháp: Mở rộng không gian tên, Tái sử dụng lại định danh, v.v

Các giải pháp không gian tên phẳng

- **Không gian tên phẳng** là một cách thiết kế định danh **đơn giản, không chứa ngữ nghĩa hay cấu trúc**.
- Nó **hỗ trợ định danh** bằng cách đảm bảo mỗi thực thể có một tên duy nhất, nhưng **không hỗ trợ định vị** (phải dùng cơ chế khác để tra cứu vị trí thực thể).
- Là chuỗi các bit, chuỗi ký tự ngẫu nhiên không cấu trúc. Không chứa bất kỳ thông tin nào về cách xác định vị trí các điểm truy cập của thực thể đã được liên kết của chúng.
- Nhiệm vụ: cho biết tên và xác định điểm truy nhập tới các thực thể.

1. Phương pháp quảng bá

•Điều kiện: hệ phân tán hỗ trợ việc trao đổi thông tin thông qua quảng bá

B1: Một thông báo có chứa định danh cần phân giải được quảng bá tới tất cả các thực thể trong hệ thống.

B2: Thực thể nào có đúng định danh trong thông báo nhận được sẽ quảng bá một thông báo chứa định danh và địa chỉ của thực thể.

B3: Tất cả các thực thể khác sẽ nhận được thông báo này và có được ánh xạ giữa định danh và địa chỉ của thực thể nói trên.

Khi cần truy xuất một thông tin nhất định, cần gửi đến cho tất cả các nút ở trong hệ thống

B1 – Quảng bá yêu cầu tìm file

- Nút A cần file có định danh **File123**.
- Nút A **gửi broadcast** yêu cầu: “Ai có file với mã **File123**?”

B2 – Nút sở hữu phản hồi

- Nút B giữ file `File123`.
- Nút B thấy định danh khớp → nó **quảng bá phản hồi**: “Tôi có `File123`, địa chỉ tôi là `192.168.1.22:5000`.”

B3 – Các nút cập nhật ánh xạ

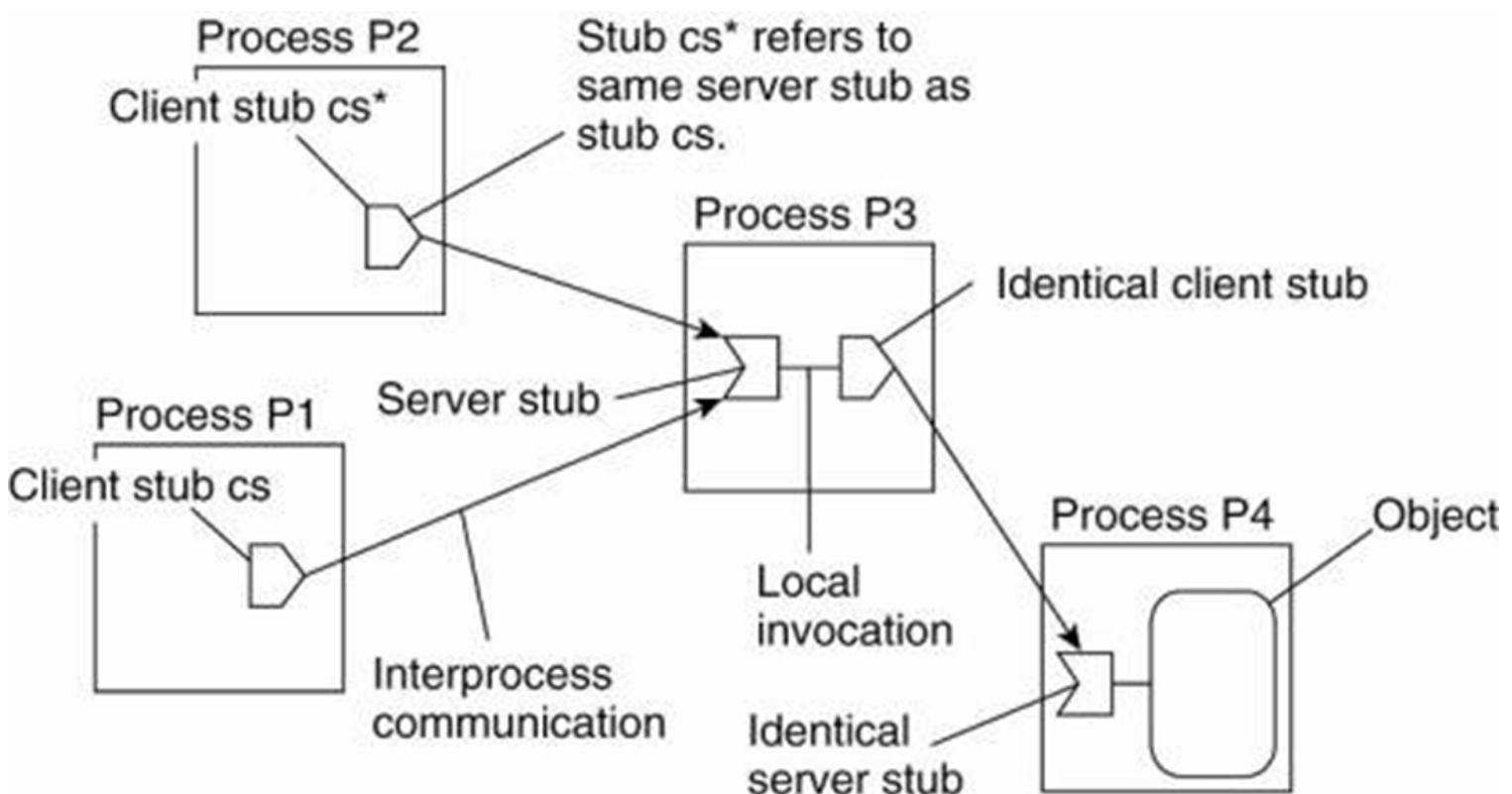
- Các nút khác trong mạng (C, D, E...) nhận được thông báo từ B.
- Mỗi nút giờ đây có ánh xạ: `File123` ↔ `192.168.1.22:5000`
- Nút A cũng ghi nhớ thông tin này để có thể kết nối trực tiếp đến B và lấy file.

2. Phương pháp chuyển tiếp con trỏ

• Phương pháp dựa trên nguyên tắc một thực thể khi rời sang vị trí khác thì phải để lại thông tin tham chiếu tới vị trí mới.

• Khi cần tìm một thực thể, client có thể xác định ngay được địa chỉ hiện tại của thực thể này nhờ phương pháp dò tìm lưu vết.

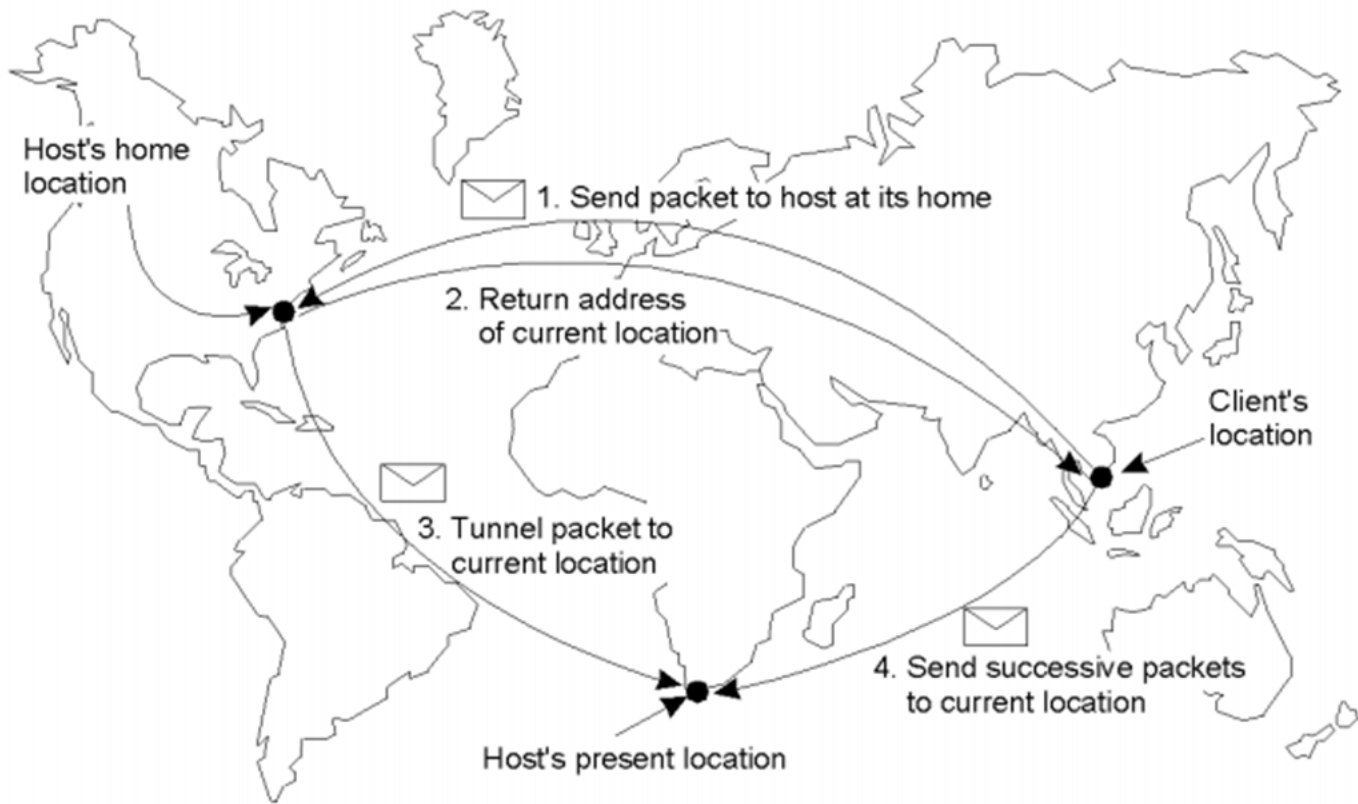
Sử dụng con trỏ để tiếp tục truy tìm thông tin trong trường hợp một thực thể rời sang vị trí khác. Ví dụ, nếu file x.pdf đang ở máy A, mà chuyển sang máy B, thì máy A sẽ có một con trỏ để trỏ sang máy B trong trường hợp cần tìm file x.pdf



Client stub: con trỏ chứa địa chỉ để chuyển tiếp đến máy tiếp theo (server stub) - để chuyển tiếp thông tin
 Server stub: con trỏ chứa địa chỉ của máy chuyển tiếp trước đó (địa chỉ client stub) - để có thể truyền ngược lại thông tin (sau khi lấy được dữ liệu)

Ví dụ trên hình, P2 có thể truy cập file bằng cách lấy địa chỉ copy từ client stub của P1, trở đến P3 -> trở đến P4 -> lấy object

3 Phương pháp Home Base



Mỗi thực thể sẽ được gắn với một home-agent (một server quản lý địa chỉ của thực thể đó)

Ví dụ trên hình:

Client gửi request đến địa chỉ của host home

Lần gửi packet đầu tiên, host home sẽ gửi đến cho vị trí host hiện tại

Host Home trả về địa chỉ hiện tại của thực thể của host

các lần gửi sau client sẽ gửi thông tin đến vị trí hiện tại của host

Tuy nhiên: trong một số trường hợp, quãng đường xa truyền đạt thông tin -> delay đường truyền, thông tin

Bảng băm phân tán (Tham Khảo trong slides bài 4, Định danh)

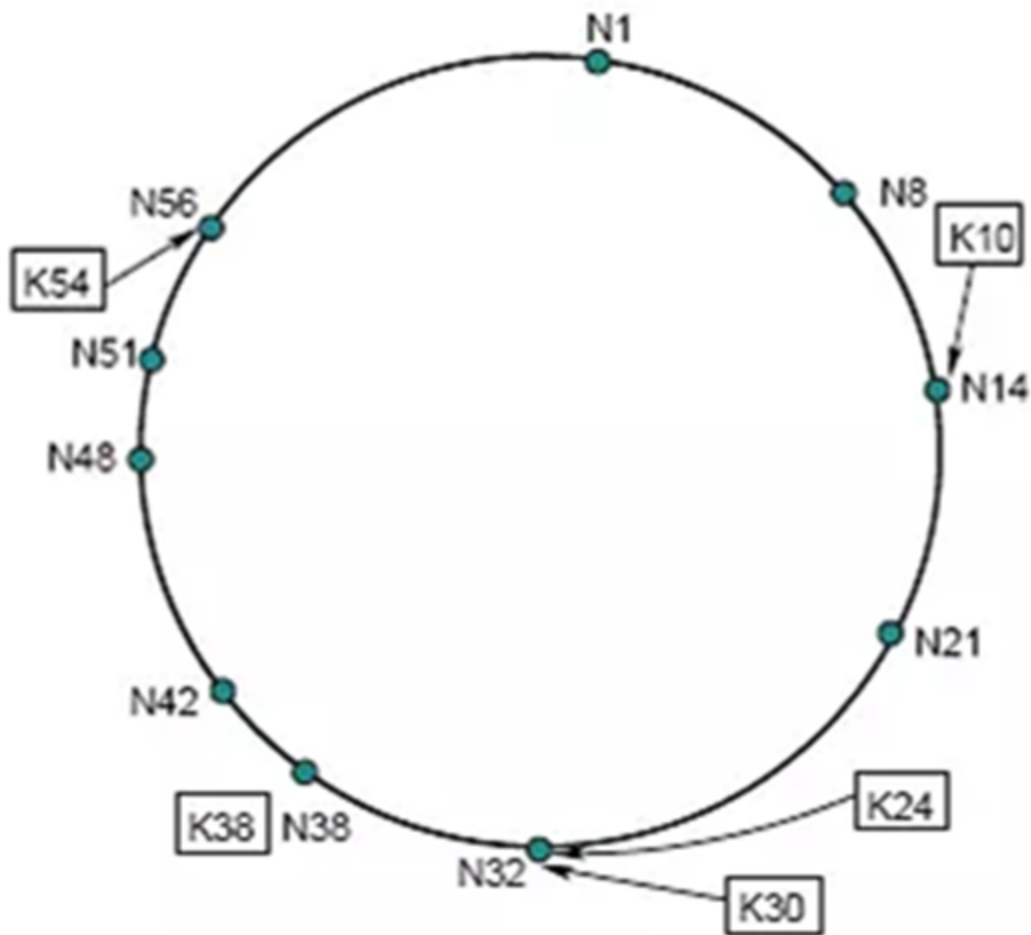
DHT = Distributed Hash Table

Hệ thống Chord:

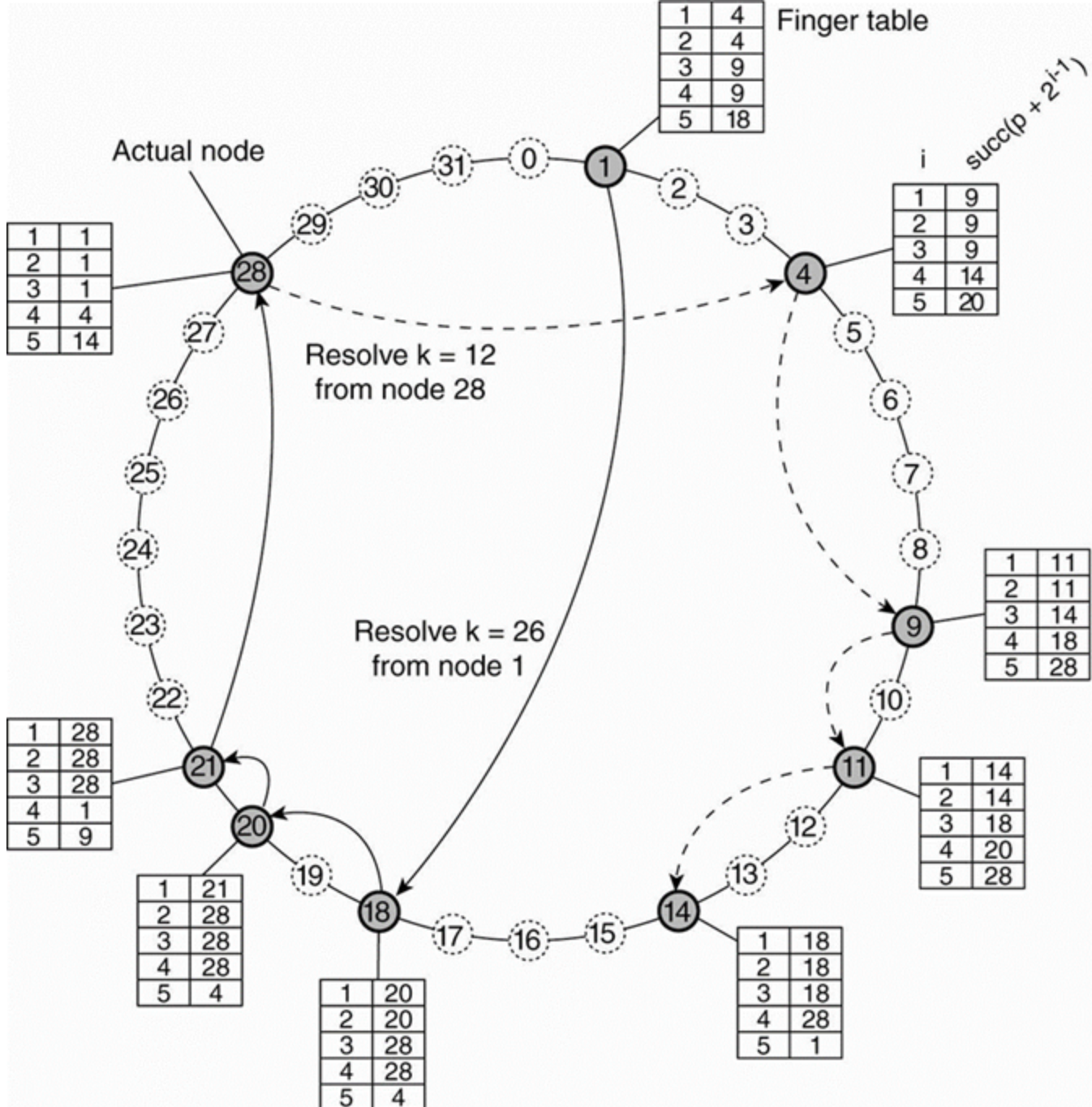
Ý tưởng chính: Gán dữ liệu (Key) cho trước với một nút mạng, sử dụng cùng **một hàm băm** để đưa dữ liệu về một giá trị, dựa vào giá trị này, nút (máy) tương ứng sẽ lưu file đó. trong một hệ thống, nút tiếp theo lớn hơn hoặc bằng sẽ đảm nhiệm nhiệm vụ lưu file đó, ví dụ

File map về giá trị 24 --> lưu vào máy 32 (trong hình)

File map về giá trị 30 --> lưu vào máy 32 (trong hình)



Tham khảo slides, các node sẽ lưu một bảng finger table để xác định các vị trí cách nó. Mục đích là để truy tìm dữ liệu nhanh hơn. lưu ở nhiều vị trí thay vì một vị trí duy nhất => dễ dàng tiếp cận trong trường hợp có một nút bị lỗi.



Giải pháp phân cấp

Lưu trữ thông tin dưới dạng cha - con giữa các nút để dễ dàng truy xuất

- [illegible]